

3650. Minimum Cost Path with Edge Reversals

Solved 

Medium

 Topics

 Companies

 Hint

You are given a directed, weighted graph with n nodes labeled from 0 to $n - 1$, and an array `edges` where `edges[i] = [ui, vi, wi]` represents a directed edge from node u_i to node v_i with cost w_i .

Each node u_i has a switch that can be used **at most once**: when you arrive at u_i and have not yet used its switch, you may activate it on one of its incoming edges $v_i \rightarrow u_i$ reverse that edge to $u_i \rightarrow v_i$ and **immediately** traverse it.

The reversal is only valid for that single move, and using a reversed edge costs $2 * w_i$.

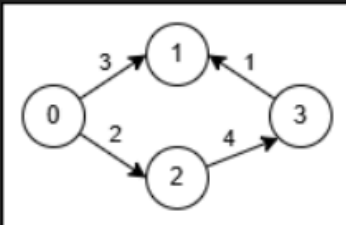
Return the **minimum** total cost to travel from node 0 to node $n - 1$. If it is not possible, return -1.

Example 1:

Input: $n = 4$, `edges = [[0,1,3], [3,1,1], [2,3,4], [0,2,2]]`

Output: 5

Explanation:



- Use the path $0 \rightarrow 1$ (cost 3).
- At node 1 reverse the original edge $3 \rightarrow 1$ into $1 \rightarrow 3$ and traverse it at cost $2 * 1 = 2$.
- Total cost is $3 + 2 = 5$.

Example 2:

Input: $n = 4$, $edges = [[0, 2, 1], [2, 1, 1], [1, 3, 1], [2, 3, 3]]$

Output: 3

Explanation:

- No reversal is needed. Take the path $0 \rightarrow 2$ (cost 1), then $2 \rightarrow 1$ (cost 1), then $1 \rightarrow 3$ (cost 1).
- Total cost is $1 + 1 + 1 = 3$.

Constraints:

- $2 \leq n \leq 5 * 10^4$
- $1 \leq edges.length \leq 10^5$
- $edges[i] = [u_i, v_i, w_i]$
- $0 \leq u_i, v_i \leq n - 1$
- $1 \leq w_i \leq 1000$

Python:

```
import heapq
```

```
class Solution:
```

```
    def minCost(self, n, edges):
```

```
        out = [[] for _ in range(n)]
```

```
        inn = [[] for _ in range(n)]
```

```
        for u, v, w in edges:
```

```
            out[u].append((v, w))
```

```
            inn[v].append((u, w))
```

```
        INF = 10**18
```

```
        dist = [[INF, INF] for _ in range(n)]
```

```
        pq = []
```

```
        dist[0][0] = 0
```

```
        heapq.heappush(pq, (0, 0, 0)) # cost, node, used
```

```
        while pq:
```

```

cost, u, used = heapq.heappop(pq)
if cost > dist[u][used]:
    continue

# Normal edges
for v, w in out[u]:
    if dist[v][0] > cost + w:
        dist[v][0] = cost + w
        heapq.heappush(pq, (dist[v][0], v, 0))

# Reversed edges (only if switch unused)
if used == 0:
    for v, w in inn[u]:
        if dist[v][0] > cost + 2 * w:
            dist[v][0] = cost + 2 * w
            heapq.heappush(pq, (dist[v][0], v, 0))

ans = min(dist[n - 1][0], dist[n - 1][1])
return -1 if ans >= INF else ans

```

JavaScript:

```

/**
 * @param {number} n
 * @param {number[][]} edges
 * @return {number}
 */
var minCost = function (n, edges) {

    const graph = edges.reduce((mapper, [from, to, cost]) => {
        mapper[from] = mapper[from] ?? [];
        mapper[from].push({ node: to, cost });

        mapper[to] = mapper[to] ?? [];
        mapper[to].push({ node: from, cost: cost * 2 });
        return mapper;
    }, {});

    const pq = new PriorityQueue((a, b) => {
        return a.cost - b.cost;
    });

    const visited = new Set();
    visited.add(0);

    for (const { node, cost } of (graph[0] ?? [])) {

```

```

    pq.enqueue({ node, cost });
}

while (!pq.isEmpty()) {
    const { node, cost } = pq.dequeue();

    if (node == n - 1) return cost;

    if (visited.has(node)) continue;

    visited.add(node);

    for (const { node: n, cost: c } of (graph[node] ?? [])) {
        pq.enqueue({ node:n, cost: c + cost });
    }
}

return -1;
};

```

Java:

```

import java.util.*;

class Solution {
    public int minCost(int n, int[][] edges) {
        List<List<int[]>> out = new ArrayList<>();
        List<List<int[]>> in = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            out.add(new ArrayList<>());
            in.add(new ArrayList<>());
        }

        for (int[] e : edges) {
            out.get(e[0]).add(new int[]{e[1], e[2]});
            in.get(e[1]).add(new int[]{e[0], e[2]});
        }

        long INF = (long) 1e18;
        long[][] dist = new long[n][2];
        for (int i = 0; i < n; i++) {
            Arrays.fill(dist[i], INF);
        }

        PriorityQueue<long[]> pq =
            new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
    }
}

```

```

dist[0][0] = 0;
pq.add(new long[]{0, 0, 0}); // cost, node, used

while (!pq.isEmpty()) {
    long[] cur = pq.poll();
    long cost = cur[0];
    int u = (int) cur[1];
    int used = (int) cur[2];

    if (cost > dist[u][used]) continue;

    // Normal edges
    for (int[] edge : out.get(u)) {
        int v = edge[0];
        int w = edge[1];
        if (dist[v][0] > cost + w) {
            dist[v][0] = cost + w;
            pq.add(new long[]{dist[v][0], v, 0});
        }
    }

    // Reversed edges (only if switch unused)
    if (used == 0) {
        for (int[] edge : in.get(u)) {
            int v = edge[0];
            int w = edge[1];
            if (dist[v][0] > cost + 2L * w) {
                dist[v][0] = cost + 2L * w;
                pq.add(new long[]{dist[v][0], v, 0});
            }
        }
    }
}

long ans = Math.min(dist[n - 1][0], dist[n - 1][1]);
return ans >= INF ? -1 : (int) ans;
}
}

```